SECTION 2 OF 3

Multi-Layer Networks, High-Dimensional Inputs & Activations

From one neuron to a network — and the functions that make it powerful

Multi-Layer Perceptrons · Images as Vectors · Activation Functions · Classification

QUICK RECAP

Where We Left Off

- A neuron computes $z = w \cdot x + b$, then applies an activation function to get its output
- A single perceptron (step activation) can only draw a straight-line decision boundary
- Problems like XOR are not linearly separable — one neuron isn't enough

Today's question

What happens when we connect many neurons together?

What You'll Be Able to Do

- Explain why stacking layers solves problems a single perceptron cannot
- Describe forward propagation through a multi-layer network, in words and in matrix form
- Turn an image (or any structured input) into a high-dimensional input vector
- Choose an appropriate activation function for a hidden layer and for an output layer
- Set up a binary or multi-class classification task and its output representation

PART A

From Perceptron to Multi-Layer Networks

Why one neuron isn't enough, and how stacking neurons fixes that

The Limit of a Single Neuron: XOR

- XOR outputs 1 only when its two binary inputs differ: (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0
- No single straight line can separate the 1s from the 0s on this pattern
- Minsky & Papert (1969) used exactly this example to show the perceptron's limits
- The fix isn't a smarter neuron — it's more neurons, arranged in layers

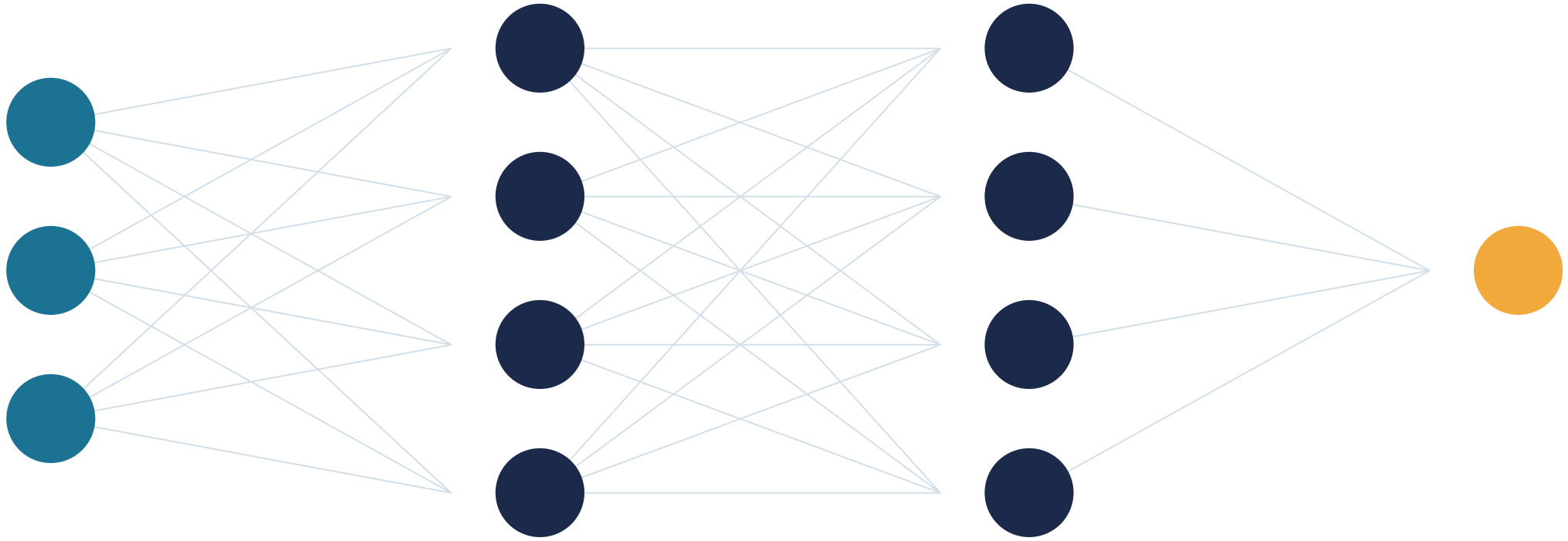
XOR truth table

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Solving It: Add a Hidden Layer

- Two neurons in a hidden layer can each learn a different straight-line split
- A third neuron combines their outputs into a curved, non-linear decision boundary
- This is the core idea of the Multi-Layer Perceptron (MLP): depth buys non-linearity
- With at least one hidden layer, a network becomes a universal function approximator

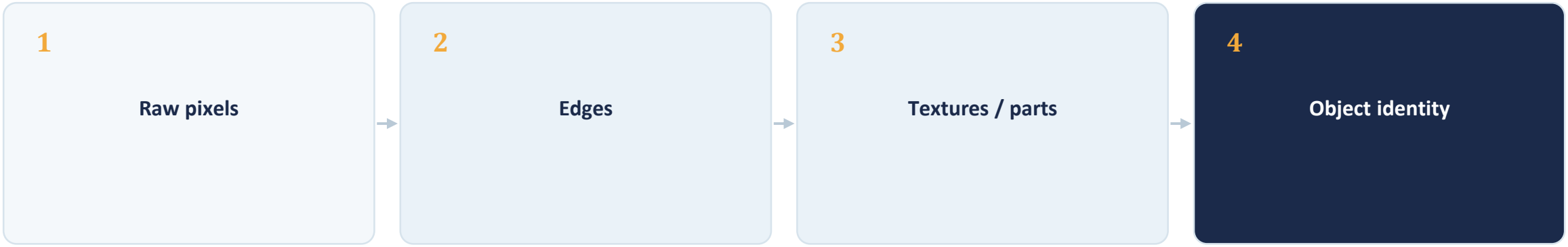
Multi-Layer Perceptron: Architecture



"Shallow" = 1–2 hidden layers. "Deep" = 3 or more. Every layer here is fully connected (Dense).

What Hidden Layers Actually Learn

- Each hidden layer re-represents its input as a new set of features
- Early layers often pick up simple patterns; later layers combine them into more abstract ones
- In an image network: edges → textures → parts → whole objects, layer by layer
- This hierarchy is learned automatically from data — nobody hand-designs these features



Shallow vs. Deep Networks

Shallow (1–2 hidden layers)

- Fewer parameters, faster to train
- Enough for many tabular / simple tasks
- Can still approximate complex functions given enough neurons

Deep (3+ hidden layers)

- Learns richer, hierarchical features
- Needed for images, speech, language
- More data & compute, more care with training

PART C

Activation Functions

The non-linear ingredient that makes deep networks powerful

Why Non-Linearity Matters

- Without an activation function, stacking layers collapses to one linear layer, no matter how deep
- A linear function can still only draw straight lines — no curved decision boundaries
- Non-linear activations let each additional layer add real modeling power
- The choice of activation affects training speed, stability, and what a layer can represent

Sigmoid

- **Formula: $\sigma(z) = 1 / (1 + e^{-z})$**
- Squashes any real number into the range (0, 1)
- Reads naturally as a probability — a common choice for binary output layers
- Downside: gradients shrink for very large |z| (the "vanishing gradient" problem)

$$\sigma(z) = 1 / (1 + e^{-z})$$

Range: (0, 1) Common use: binary classification output

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

sigmoid(1.4)
# -> 0.802
```

Tanh

- **Formula: $\tanh(z) = (e^{z} - e^{-z}) / (e^{z} + e^{-z})$**
- Squashes values into the range (-1, 1) — zero-centered, unlike sigmoid
- Zero-centered outputs often help hidden layers train a little faster
- Still suffers from vanishing gradients for large |z|, just like sigmoid

$$\tanh(z) = (e^{z} - e^{-z}) / (e^{z} + e^{-z})$$

Range: (-1, 1) Common use: zero-centered hidden layers

```
def tanh(z):
    return np.tanh(z)

tanh(1.4)
# -> 0.885
```

ReLU & Leaky ReLU

- **Formula: ReLU(z) = max(0, z)**
- Cheap to compute, and gradients don't shrink for positive z — trains fast
- The default choice for hidden layers in most modern networks
- Downside: a neuron stuck with z < 0 forever stops learning ("dying ReLU")
- Leaky ReLU fixes this with a small slope for negative z instead of a hard 0

ReLU(z) = max(0, z)

Leaky ReLU(z) = max(0.01z, z)

Range: [0, ∞) Common use: hidden layers (default choice)

```
relu = np.maximum(0, z)
# vectorized
```

Choosing the Right Activation

Layer / Task	Typical Choice	Why
Hidden layers	`ReLU`	Fast, effective default; try Leaky ReLU if neurons "die"
Binary output	`Sigmoid`	One unit, output read directly as a probability
Multi-class output	`Softmax`	One unit per class, outputs sum to 1
Regression output	`None (linear)`	Let the raw weighted sum be the prediction

Binary Classification

- The network must choose between exactly two classes — e.g. "cat" vs. "not cat"
- Output layer: a single unit with a sigmoid activation
- A prediction above 0.5 is read as the positive class, otherwise the negative class
- Trained with binary cross-entropy loss (more on this in Section 3)

```
output = sigmoid(z)      class = 1 if output > 0.5 else 0
```